# 第二章 数据的表示和运算

2022年9月7日 星期三 19:55

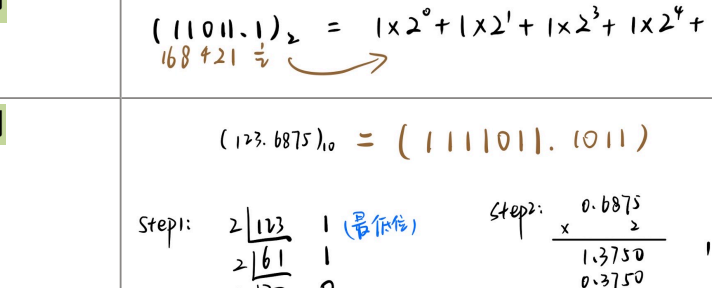
问题1：为什么计算机要用二进制来表示数据呢?
回答：计算机是通过CPU来进行计算的，CPU上有很多引脚，引脚有电流通过。
当电流通过时表示数值1，不通过时表示数值0。由于硬件的限制，只能用0和1这样的二进制来表示数据。

问题2：只有0和1就表示数字就够了吗？0，1，下一位是什么呢?
回答：我们平常使用的是十进制，是因为我们有10个手指头，从0开始数到9的时候手指就用完了，再往前数就记下来进一个10
计算机使用的是二进制，数到1之后就没有值可以表示了，数完1就进一位继续数。

问题3：计算机计算小数为什么有可能出错？如C里面连续累加100个0.1，得到的float类型的值为10.000002
回答：因为有一些十进制的小数无法转换成二进制，如十进制的0.1就无法用二进制表示，二进制只能表示的小数为$\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{16}$...的组合
实际二进制无法表示0.1就类似于十进制无法表示$\frac{1}{3}$，他们都是循环小数。

## 数制

- 字节是最基本的信息单位【B】
- 位是最小的信息单位【bit】
- 1字节(B)=8位(bit)
- 用二进制编码的好处
  - 二进制的运算规则简单
  - 制造两个稳态的物理器间较容易
  - 便于用逻辑门电路实现算术运算

<table>
<tr><th>进位计数制</th><th>数制的相互转换</th></tr>
<tr><td>
• 十进制【Decimal system】【D】<br>
• 二进制【Binary】【B】<br>
  ○ 如101.11=5.75<br>
• 八进制【Octal number system】【O】<br>
  ○ 把二进制的三位一组就是八进制<br>
• 十六进制【Hexadecimal】【H】<br>
  ○ 把二进制的四位一组就是十六进制<br>
  ○ 范围：0~9+ABCDEF
</td><td>
<table>
<tr><td>二进制--->八进制/十六进制</td><td>$(1111000010.01101)_2 = (1702.32)_8$<br>001 111 000 010 . 011 010<br>1 7 0 2 . 3 2<br>$(1111000010.01101)_2 = (3C2.68)_{16}$<br>0011 1100 0010 . 0110 1000<br>3 C 2 . 6 8</td></tr>
<tr><td>任意进制--->十进制</td><td>$(11011.1)_2 = 1\times2^0+1\times2^1+1\times2^3+1\times2^4+1\times2^{-1}=27.5$<br>168 421 ½</td></tr>
<tr><td>十进制--->任意进制</td><td>$(123.6875)_{10} = (1111011.1011)$<br>

Step1:<br>2|123 1 (最低位)<br>2|61 1<br>2|30 0<br>2|15 1<br>2|7 1<br>2|3 1<br>2|1 1 (最高位)<br>0<br>Step2:<br>0.6875 ×2 1.3750 1 (最高位)<br>0.3750 ×2 0.7500 0<br>×2 1.5000 1<br>0.5000 ×2 1.0000 1 (最低位)

</td></tr>
</table>
</td></tr>
</table>

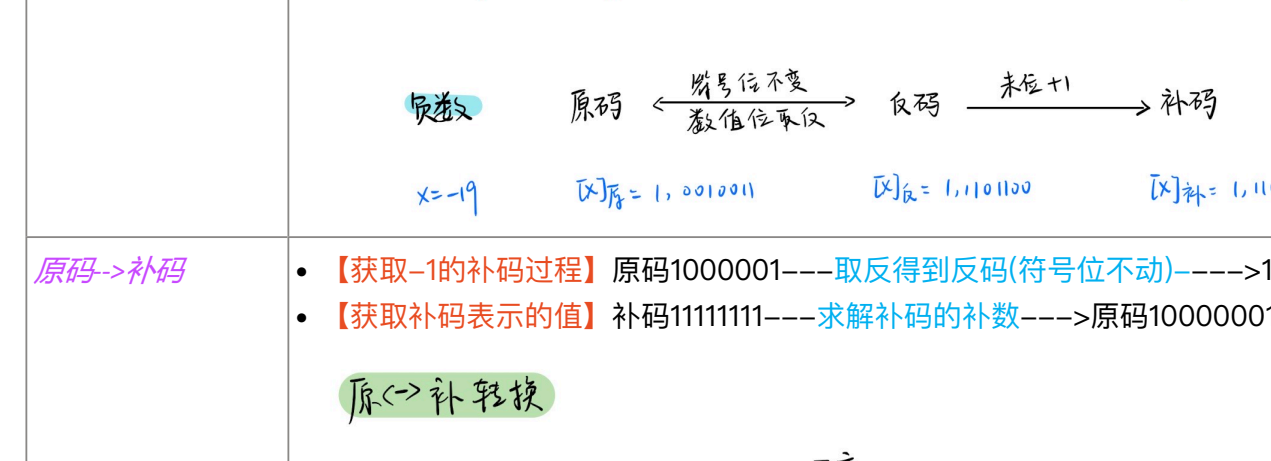
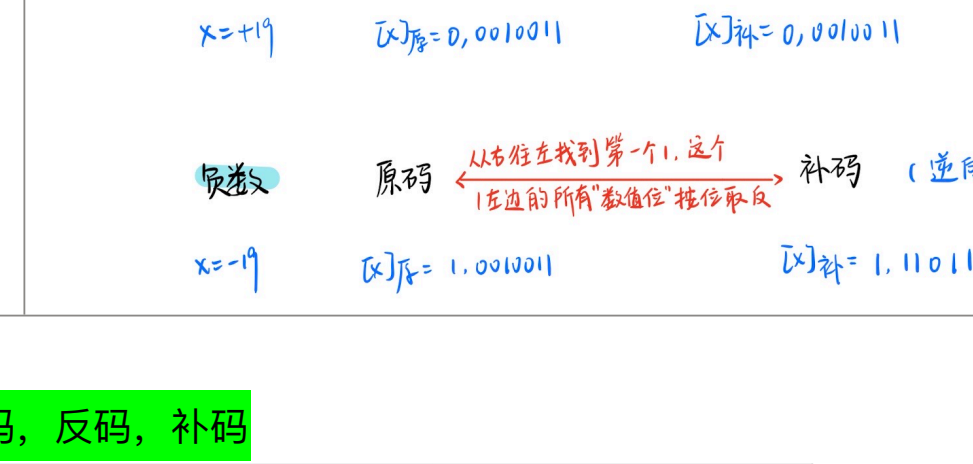
## 无符号数

- 寄存器的位数决定了无符号数的表示范围。如8位寄存器，无符号数表示范围为0-255
- 用无符号数表示主存地址
- 8位无符号数的范围：0~255
- 16位无符号数的范围：0~65535

## 有符号数

| | |
|---|---|
| 真值和机器数 | • 真值：+5<br>• 机器数：05【0表示正，1表示负】 |
| 原码，反码，补码 | 如果正负数都用原码表示，则出现如下情况<br>• 1的原码00000001，−1的原码10000001，则1+(−1)=00000001+10000001=10000010，结果不是0<br>• 所以用原码可以表示正数，但是不能表示负数<br>为了表示负数，引入补码【二进制的补数】<br>• 补码重要法则：(二进制的值取反+1) + (原来的值) = 0<br>• 【获取−1的补码过程】原码1000001---取反得到反码(符号位不动)---->11111110---加1得到补码---->11111111<br>• 【获取补码表示的值】补码11111111---求解补码的补码--->原码10000001(对应二进制−1)<br>无符号数和有符号数表示的范围【这是补码的表示范围】<br>• 8位补码表示的整数范围是−128~127【常考】<br>• short和unsigned short都是2字节(16位)的变量<br>• 【有符号数】short表示范围：−32768~32767（$-2^{n-1}\sim2^{n-1}-1$）<br>  ○ 为什么负数比正数多一?<br>    ▪ 最高位为0的正数，有0~32767共32768个【包括0】<br>    ▪ 最高位为1的负数，有−1~−32768共32768个【不包括0】<br>• 【无符号数】unsigned shorty表示范围：0~65535（$2^{16}-1=65535$）<br>补码的优点<br>• 0的补码表示唯一，都是0000 0000<br>• 补码运算规则简单，且符号位可以和数值位一起参加运算<br>• 补码比原码和反码多表示一个最小负数（−128） |
| BCD码 | 【8421码】有权码<br>• 四位数权值分别为8，4，2，1<br>• 2个8421码加的结果大于9（1001），则需要修正，结果加一个6（0110）<br>• 如8--->1000, 9---->1001<br>【余3码】无权码<br>• 在8421的基础上加3（0011）<br>• 如8---->1011, 9---->1100<br>【2421码】有权码<br>• 四位数权值分别为2，4，2，1<br>• 大于等于5的4位二进制中最高位为1，小于5的最高位为0<br>• 如5--->1011而不是0101 |
| 移码 | • 移码常用来表示浮点数的阶码，只能表示整数<br>• 整数的补码和移码符号位相反，数值位相同<br>• $[x]_移 = 2^n + x\ (2^n > x \geq -2^n$, 机器字长为$n+1)$<br>• 如$x_1 = +10101$, 字长为8位，$[x_1]_移 = 2^7 + 10101 = 10010101$<br>• 如$x_2 = -10101$, 字长为8位，$[x_2]_移 = 2^7 + (-10101) = 01101011$ |

## 码之间的转换

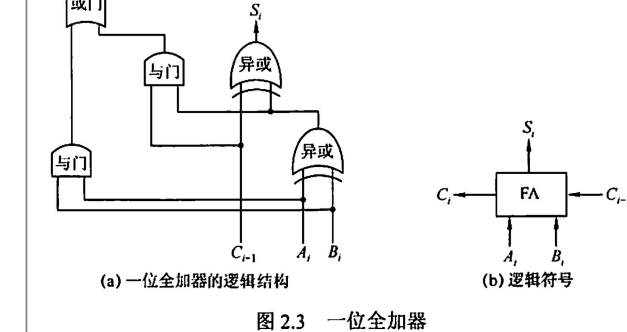
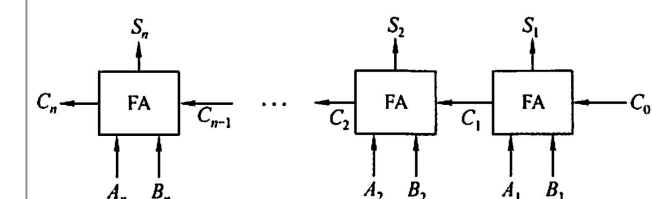
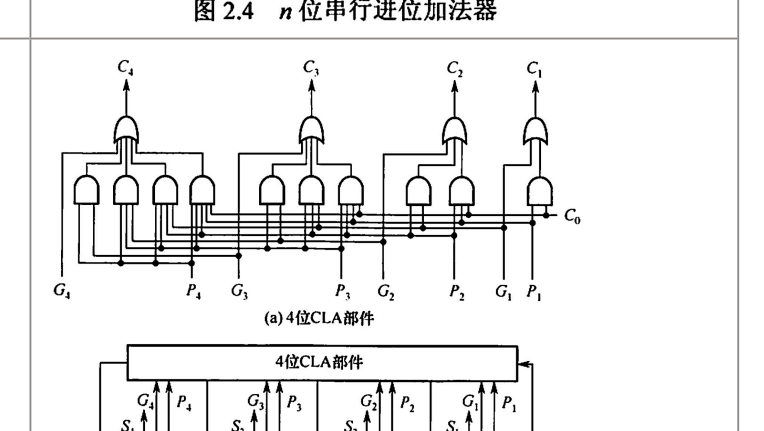
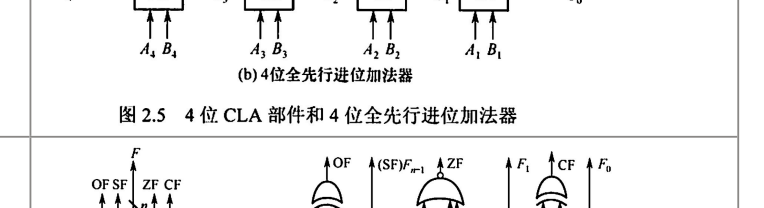
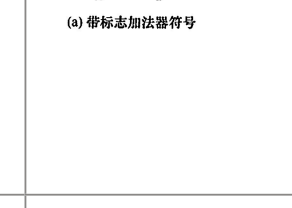
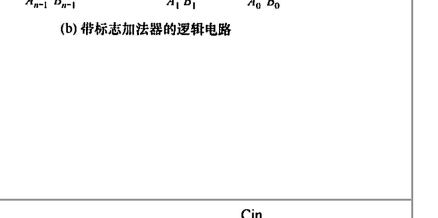
| | |
|---|---|
| 原码-->反码-->补码 |  原>反>补转换<br>正数<br>(更便于人类理解)<br>原码<br>不变<br>(中间量)<br>反码<br>不变<br>(更便于计算机运算)<br>补码<br>x=+19<br>[x]原=0,0010011<br>[x]反=0,0010011<br>[x]补=0,0010011<br>负数<br>原码<br>符号位不变<br>数值位取反<br>反码<br>末位+1<br>补码<br>x=−19<br>[x]原=1,0010011<br>[x]反=1,1101100<br>[x]补=1,1101101 |
| 原码-->补码 | • 【获取−1的补码过程】原码1000001---取反得到反码(符号位不动)---->11111110---加1得到补码---->11111111<br>• 【获取补码表示的值】补码11111111---求解补码的补数--->原码10000001(对应二进制−1)<br> 原<->补转换<br>正数<br>原码<br>不变<br>补码<br>x=+19<br>[x]原=0,0010011<br>[x]补=0,0010011<br>负数<br>原码<br>从右往左找到第一个1，这个1左边的所有"数值位"按位取反<br>补码<br>(逆向也适合)<br>x=−19<br>[x]原=1,0010011<br>[x]补=1,1101101 |

## 常考的数值的原码，反码，补码

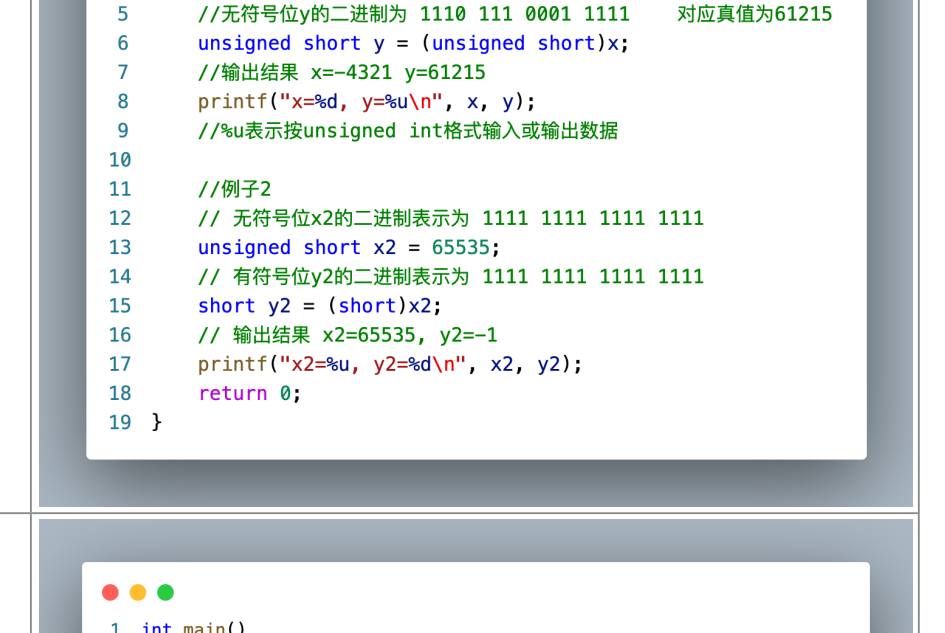
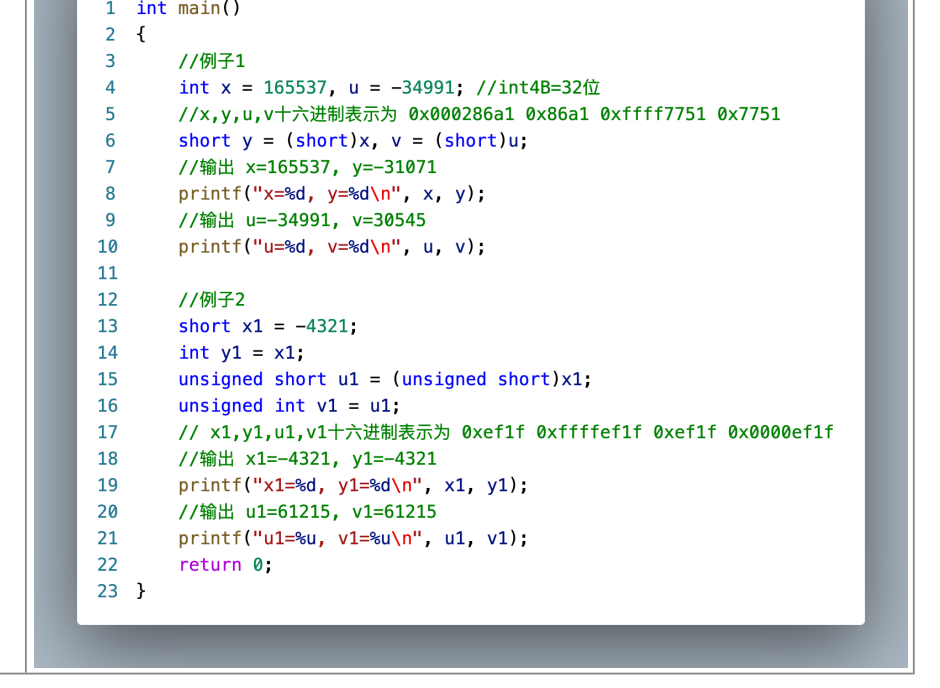
| | 原码 | 反码 | 补码 | 移码 |
|---|---|---|---|---|
| 1 | 0000 0001 | 0000 0001 | 0000 0001 | 1000 0001 |
| -1 | 1000 0001 | 1111 1110 | 1111 1111 | 0111 1111 |
| 0 | 0000 0000 | 0000 0000 | 0000 0000 | 1000 0000 |
| -0【-0等同于-128】 | | | 1000 0000 | 0000 0000 |
| +127 | 0111 1111 | 0111 1111 | 0111 1111 | 1111 1111 |
| -128 | 无 | 无 | 1000 0000 | 0000 0000 |

## 基本运算部件

- 补充逻辑运算的知识
  - 【逻辑非NOT】
  - 【逻辑与AND】只有全为1值才为1
  - 【逻辑或OR】有一个为1值就为1
  - 【逻辑异或XOR】两个值相同就为1
  - 逻辑运算的对象不是数值，因此不会出现进位的情况
  - 算术逻辑单元ALU的核心部件是加法器

| | 基本概念和考点 | 图 |
|---|---|---|
| 一位全加器【FA】 | • FA是最基本的加法单元<br>• input：加数$A_i$加数$B_i$低位传来的进位$C_{i-1}$<br>• output：本位和$S_i$，向高位的进位$C_i$<br>• 和表达式 $= A_i \oplus B_i \oplus C_i$<br>• 进位表达式 $C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}$ |  $C_i$<br>或门<br>与门<br>异或<br>$S_i$<br>与门<br>异或<br>$C_{i-1}$ $A_i$ $B_i$<br>FA<br>(a) 一位全加器的逻辑结构<br>(b) 逻辑符号<br>图 2.3 一位全加器 |
| 串行进位加法器 | • 串行进位加法器：把n个FA相连得到的n位加法器<br>• 每级进位直接依赖于前一级的进位，进位信号是逐级形成的<br>• 最长运算时间主要由进位信号的传递时间决定 |  $S_n$ $S_2$ $S_1$<br>$C_n$ FA $C_{n-1}$ … $C_2$ FA $C_1$ FA $C_0$<br>$A_n$ $B_n$ $A_2$ $B_2$ $A_1$ $B_1$<br>图 2.4 n 位串行进位加法器 |
| 并行进位加法器 | • 进位信号$G_i = A_iB_i$，进位传递信号$P_i = A_i \oplus B_i$<br>• 进位表达式 $C_i = G_i + P_iC_{i-1}$<br>• $C_i$ 仅与$A_i$, $B_i$和最低进位$C_0$有关，相互间的进位没有依赖关系<br>• CLA（超前进位部件）是实现上述表达式的部件<br>• 这种进位方式快速，与位数无关，当位数多时采用全先行进位是不现实的<br>• 采用并行进位的目的是：提高加法器运算速度<br>• 通常采用两级或多级先行进位加法器 |  <br>图 2.5 4 位 CLA 部件和 4 位全先行进位加法器 |
| 带标志加法器 | • 不仅能计算和/差，还能生成相应的标志信息<br>• 为了加快加法运算的速度，实际电路一定使用多级先行进位方式<br>• 【OF】溢出标志<br>• 【SF】符号标志<br>• 【ZF】零标志，当F=0时，ZF=1<br>• 【CF】进位/借位标志，$C_{in}=0$时，$CF = C_{out}$；$C_{in}=1$时，$CF = C_{out}$取反<br>• A−B<0时，CF=1；溢出时OF=1 |  OF SF ZF CF<br>加法器<br>Cout Cin<br>A B<br>(a) 带标志加法器符号<br>(b) 带标志加法器的逻辑电路 |
| 算术逻辑单元ALU | • ALU的核心是带标志加法器<br>• ALU是功能强大的组合逻辑电路，能进行算术运算，逻辑运算，移位操作<br>• ALUop是操作控制端，决定ALU所执行的处理操作<br>• ALUop的位数决定了操作的种类，位数为3时，有$2^3=8$种操作<br>• 在ALUop的控制下，由一个多路选择器MUX选择输出某种操作结果<br>• MUX是多路选择开关，它从多个输入信号中选择一个送到输出端 |  <br>图 2.7 ALU 的基本结构<br> Cin<br>ALUop<br>A<br>and<br>or<br>add<br>一位全加器<br>MUX<br>F<br>B<br>Cout<br>图 2.8 一位 ALU 的结构 |

## C语言中的整数类型与类型转换【常考！】

| | 定义 | 实例 |
|---|---|---|
| 有符号数和无符号数的转换 | • 强制类型转换的结果保持位置不变，仅改变了解释这些位的方式 |  |
| 不同字长整数之间的转换 | • 当大字长变量向小字长变量强制类型转换时<br>系统把多余的高位部分直接截断，低位直接赋值<br>• 短字长整数到长字长整数的转换时<br>要使相应的位值相等，也要对高位部分进行填充<br>原数位无符号整数，填充0<br>原数位为有符号整数，进行符号填充<br>• char为8位无符号整数，转换为int时高位补0即可 |  |

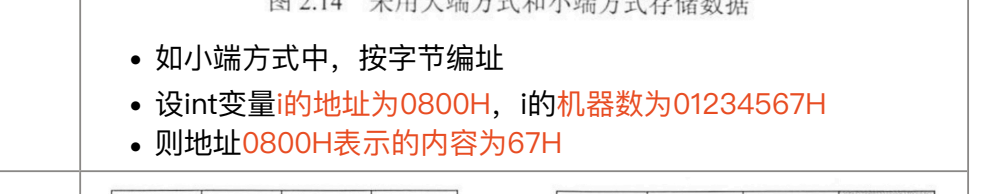
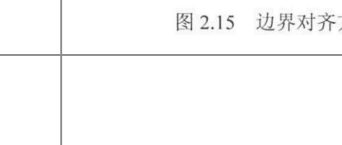
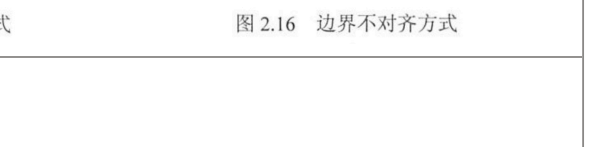
有符号数和无符号数的转换

```
int main(){
    //例子1
    //x的补码为          1110 1111 0001 1111
    short x = -4321;
    //无符号位y的二进制为 1110 111 0001 1111    对应真值为61215
    unsigned short y = (unsigned short)x;
    //输出结果 x=-4321 y=61215
    printf("x=%d, y=%u\n", x, y);
    //%u表示按unsigned int格式输入或输出数据

    //例子2
    // 无符号位x2的二进制表示为 1111 1111 1111 1111
    unsigned short x2 = 65535;
    // 有符号位y2的二进制表示为 1111 1111 1111 1111
    short y2 = (short)x2;
    // 输出结果 x2=65535, y2=-1
    printf("x2=%u, y2=%d\n", x2, y2);
    return 0;
}
```

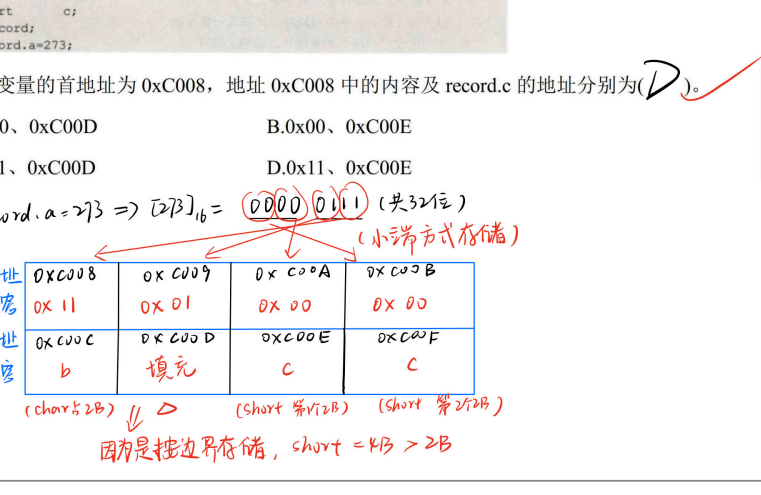
不同字长整数之间的转换

```
int main()
{
    //例子1
    int x = 165537, u = -34991; //int4B=32位
    //x,y,u,v十六进制表示为 0x000286a1 0x86a1 0xffff7751 0x7751
    short y = (short)x, v = (short)u;
    //输出 x=165537, y=-31071
    printf("x=%d, y=%d\n", x, y);
    //输出 u=-34991, v=30545
    printf("u=%d, v=%d\n", u, v);

    //例子2
    short x1 = -4321;
    int y1 = x1;
    unsigned short u1 = (unsigned short)x1;
    unsigned int v1 = u1;
    // x1,y1,u1,v1十六进制表示为 0xef1f 0xffffef1f 0xef1f 0x0000ef1f
    //输出 x1=-4321, y1=-4321
    printf("x1=%d, y1=%d\n", x1, y1);
    //输出 u1=61215, v1=61215
    printf("u1=%u, v1=%u\n", u1, v1);
    return 0;
}
```

## 数据的存储和排列【常考！重点！】

| | | |
|---|---|---|
| 数据的大端方式和小端方式存储 | • 用最低有效字节LSB和最高有效字节MSB分别表示数的低位和高位<br>• short占2字节=16位<br>• int，float占4字节=32位<br>• double占8字节=64位<br>• 大端方式：按MSB到LSB的顺序存储数据<br>• 小端方式：按LSB到MSB的顺序存储数据 |  大端方式<br>0800H 0801H 0802H 0803H<br>… 01H 23H 45H 67H …<br>小端方式<br>0800H 0801H 0802H 0803H<br>… 67H 45H 23H 01H …<br>图 2.14 采用大端方式和小端方式存储数据<br>• 如小端方式中，按字节编址<br>• 设int变量i的地址为0800H，i的机器数为01234567H<br>• 则地址0800H表示的内容为67H |
| 数据按边界对齐方式存储 | • 边界对齐相对于边界不对齐是一种空间换时间的思想<br>• 存储字长为32位时，半字是2的整数倍，字地址是4的整数倍，字节大小为8位<br>• 边界对齐虽然浪费了一些存储空间，但是可以提高取指令和取数的速度<br>• 边界不对齐可以充分利用存储空间，但是效率不太行 |  <br>图 2.15 边界对齐方式<br> 字节1 字节2 字节3 半字1-1<br>半字1-2 半字2 半字3-1<br>半字3-2 字1-1<br>字1-2<br>图 2.16 边界不对齐方式 |

例题



34.【2012 统考真题】某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct{
    int    a;
    char   b;
    short  c;
}record;
record.a=273;
```

若 record 变量的首地址为 0xC008，地址 0xC008 中的内容及 record.c 的地址分别为（D）。

A.0x00、0xC00D　　B.0x00、0xC00E

C.0x11、0xC00D　　D.0x11、0xC00E

record.a=273 ⇒ $[273]_{16}$ = 0000 0111（共32位）

（小端方式存储）

| | | | | |
|---|---|---|---|---|
| 地址 | 0xC008 | 0xC009 | 0xC00A | 0xC00B |
| 内容 | 0x 11 | 0x 01 | 0x 00 | 0x 00 |
| 地址 | 0xC00C | 0xC00D | 0xC00E | 0xC00F |
| 内容 | b | 填充 | c | c |

(char占1B) ⇓ D (short 第1/2B) (short 第2/2B)

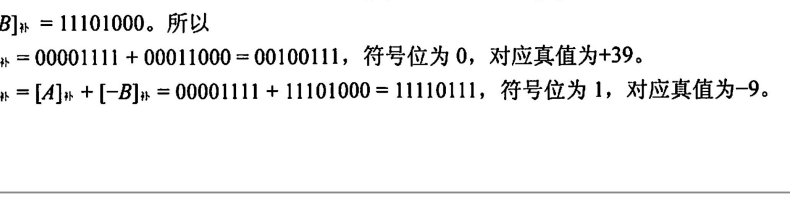
因为是按边界存储，short = 4B > 2B

## 定点数

- 位数不发生移动的数
- 运算过程中可以不考虑定点数是小数还是整数

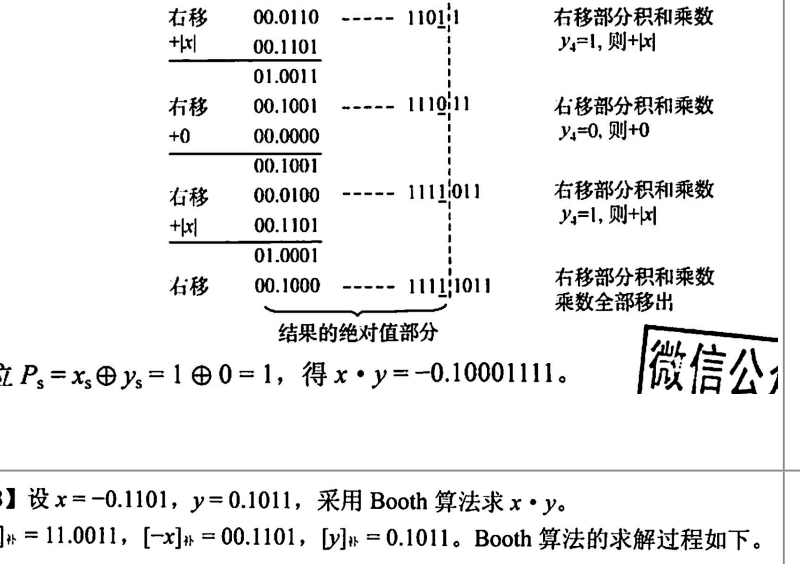
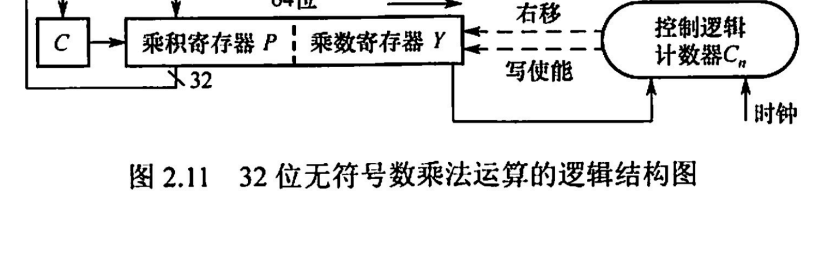
## 定点数的运算

### 加减法【常考加减法】

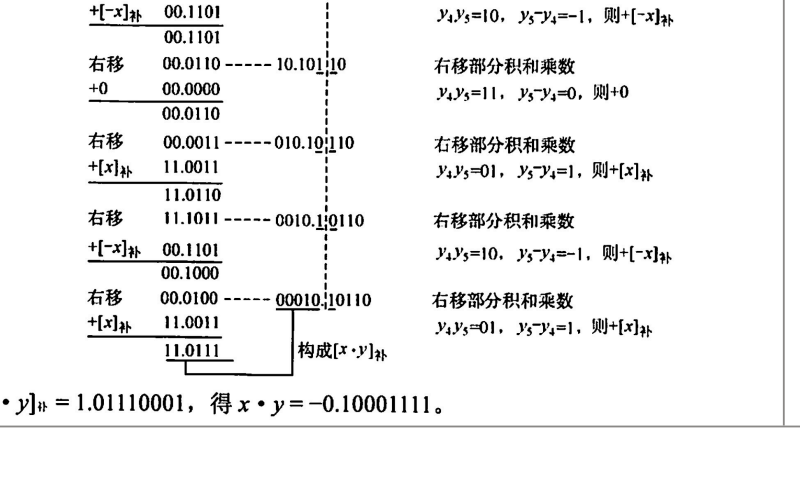
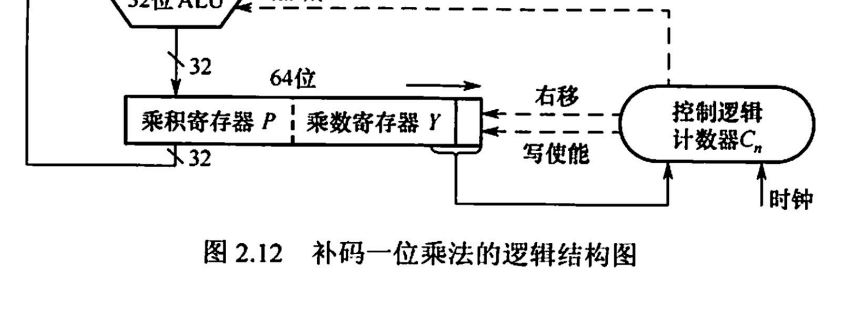
<table>
<tr><th></th><th>定义和相关概念</th><th>实例</th><th>运算电路信号解释</th></tr>
<tr><td>补码的加减法运算</td><td>• 按二进制运算规则就可以了<br>• 符号位和数值位一起参与运算<br>• 运算结果的高位丢弃</td><td><br>【例 2.6】设机器字长为 8 位（含 1 位符号位），A = 15，B = 24，求[A + B]补和[A − B]补。<br>解：<br>A = +15 = +0001111，B = +24 = +0011000；得[A]补 = 00001111，[B]补 = 00011000。<br>求得[−B]补 = 11101000。所以<br>[A + B]补 = 00001111 + 00011000 = 00100111，符号位为 0，对应真值为+39。<br>[A − B]补 = [A]补 + [−B]补 = 00001111 + 11101000 = 11110111，符号位为 1，对应真值为−9。</td><td>
<table>
<tr><th>记忆方法</th><th>解释</th><th>是否有意义</th></tr>
<tr><td>Zero零</td><td>ZF=1表示结果F为0</td><td>对无符号和有符号数都有意义</td></tr>
<tr><td>Overflow溢出</td><td>OF=1表示带符号整数运算发生溢出</td><td>对无符号数没有意义</td></tr>
<tr><td>Signal信号</td><td>SF表示结果的符号</td><td>对无符号数没有意义</td></tr>
<tr><td>Carry进位</td><td>CF表示无符号整数运算时的进位，判断是否发生溢出</td><td>对于带符号数运算无意义</td></tr>
</table>
</td></tr>
<tr><td>溢出判别方式【重点】</td><td>• 一般用异或门来实现溢出判断电路<br>• 仅当两个符号相同的数相加或两个符号相异的数相减时才可能产生溢出<br>• 计算时，左边出现溢出，将溢出位丢掉<br>• 判断溢出的方式【右边三种方法】</td><td>1、采用一位符号法<br>○ 参与操作的两个数符号相同，结果又与原操作数符号不同，则溢出<br>2、采用一位符号位根据数据位的进位判断情况判断溢出<br>○ 若符号位的进位和最高数位的进位相同，则没有溢出</td><td>3、采用双符号法/模4补码<br>○ 模4补码有模2补码的所有优点，且更容易检查溢出问题<br>○ 模4补码存储时只需要一个符号位<br>○ 模4补码计算时（在ALU中）需要两个符号位<br>○ 最高位代表真正的符号，低位符号参与移位操作<br>○ 00表示正数，无溢出<br>○ 01表示正溢出<br>○ 10表示负溢出<br>○ 11表示负数，无溢出</td></tr>
</table>

### 乘法【不怎么考计算过程，易考概念】

- 乘法运算由累加和右移操作实现

<table>
<tr><th></th><th>相关定义和概念</th><th>计算过程</th><th>电路图</th></tr>
<tr><td>原码一位乘法</td><td>• 符号位和数值位是分开求的<br>• 符号位不参与运算，同号为正，异号为负<br>• 乘积符号 = 两个数的符号位的异或<br>• 乘积数值 = 两个数的绝对值相乘的结果<br>• 积和被乘数采用双符号位<br>• 【最多进行n次加法运算，n次移位】</td><td><br>【例 2.7】设 x = −0.1101，y = 0.1011，采用原码一位乘法求 x·y。<br>解：|x| = 00.1101，|y| = 00.1011，原码一位乘法的求解过程如下。<br>(高位部分积) (低位部分积/乘数) 说明<br>00.0000 1011 丢失位 起始情况<br>+|x| 00.1101 $y_4$=1，则+|x|<br>00.1101<br>右移 00.0110 ----- 1101 1 右移部分积和乘数<br>+|x| 00.1101 $y_4$=1，则+|x|<br>01.0011<br>右移 00.1001 ----- 1110 11 右移部分积和乘数<br>+0 00.0000 $y_4$=0，则+0<br>00.1001<br>右移 00.0100 ----- 1111 011 右移部分积和乘数<br>+|x| 00.1101 $y_4$=1，则+|x|<br>01.0001<br>右移 00.1000 ----- 1111 1011 右移部分积和乘数 乘数全部移出<br>结果的绝对值部分<br>符号位 $P_s = x_s \oplus y_s = 1 \oplus 0 = 1$，得 x·y = −0.10001111。</td><td>

被乘数寄存器 X<br>32<br>32<br>32位 ALU<br>加<br>32<br>64位<br>C<br>乘积寄存器 P<br>乘数寄存器 Y<br>右移<br>写使能<br>控制逻辑 计数器$C_n$<br>32<br>时钟

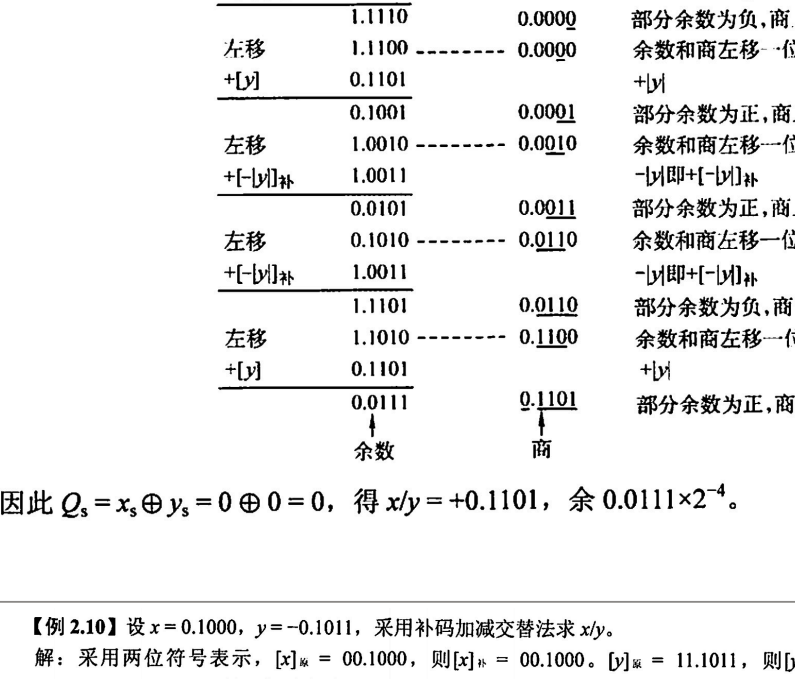
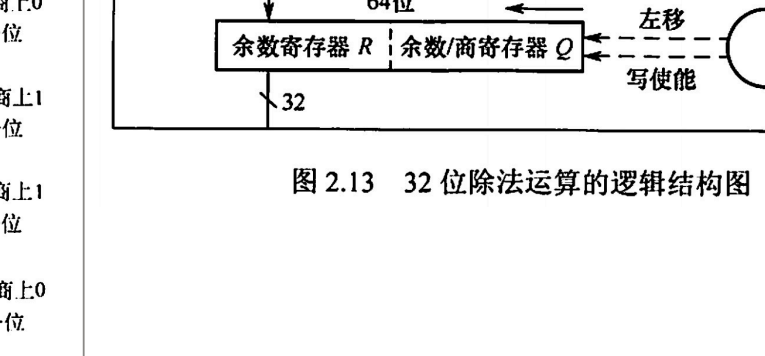
<br>图 2.11 32 位无符号数乘法运算的逻辑结构图</td></tr>
<tr><td>补码一位乘法【Booth算法】</td><td>• 是一种有符号数的乘法<br>• 【最多进行n次移位，n+1次加法运算】<br>• Booth算法的移位规则<br>
<table>
<tr><th>高位</th><th>低位</th><th>操作</th></tr>
<tr><td>0</td><td>0</td><td>部分积右移一位</td></tr>
<tr><td>0</td><td>1</td><td>部分积加[X]补，右移一位</td></tr>
<tr><td>1</td><td>0</td><td>部分积加[−X]补，右移一位</td></tr>
<tr><td>1</td><td>1</td><td>部分积右移一位</td></tr>
</table>
</td><td><br>【例 2.8】设 x = −0.1101，y = 0.1011，采用 Booth 算法求 x·y。<br>解：[x]补 = 11.0011，[−x]补 = 00.1101，[y]补 = 0.1011。Booth 算法的求解过程如下。<br>(高位部分积) (低位部分积/乘数) 说明<br>00.0000 0.1011 0 丢失位 起始情况<br>+[−x]补 00.1101 $y_4y_5$=10，$y_5-y_4$=−1，则+[−x]补<br>00.1101<br>右移 00.0110 ----- 10.101 10 右移部分积和乘数<br>+0 00.0000 $y_4y_5$=11，$y_5-y_4$=0，则+0<br>00.0110<br>右移 00.0011 ----- 010.10 110 右移部分积和乘数<br>+[x]补 11.0011 $y_4y_5$=01，$y_5-y_4$=1，则+[x]补<br>11.0110<br>右移 11.1011 ----- 0010.1 0110 右移部分积和乘数<br>+[−x]补 00.1101 $y_4y_5$=10，$y_5-y_4$=−1，则+[−x]补<br>00.1000<br>右移 00.0100 ----- 00010 10110 右移部分积和乘数<br>+[x]补 11.0011 $y_4y_5$=01，$y_5-y_4$=1，则+[x]补<br>11.0111 构成[x·y]补<br>所以[x·y]补 = 1.01110001，得 x·y = −0.10001111。</td><td>

被乘数寄存器 X<br>32<br>32<br>32位 ALU<br>加/减<br>32<br>64位<br>乘积寄存器 P<br>乘数寄存器 Y<br>右移<br>写使能<br>控制逻辑 计数器$C_n$<br>32<br>时钟

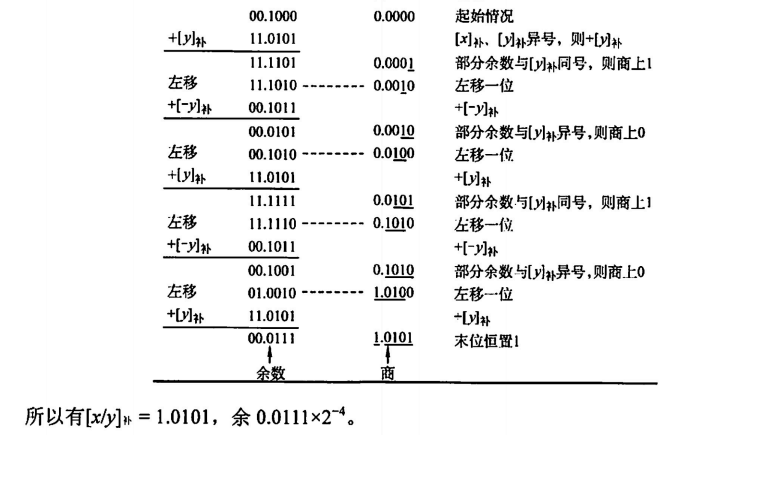
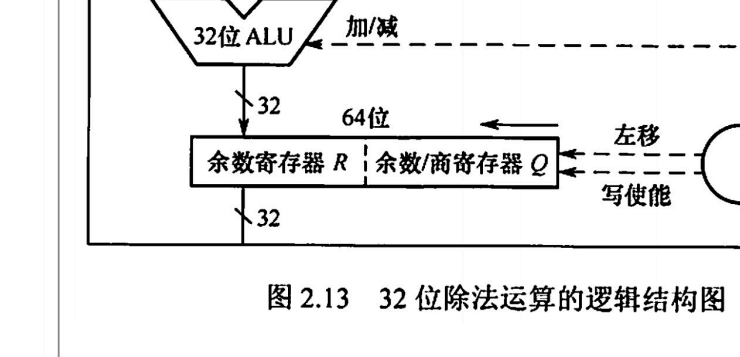
<br>图 2.12 补码一位乘法的逻辑结构图</td></tr>
</table>

### 除法【不怎么考计算过程】

- 除法运算由累加和左移（逻辑左移）实现

<table>
<tr><th></th><th>相关概念和定义</th><th>计算过程</th><th>电路图</th></tr>
<tr><td>原码除法运算【不恢复余数法】</td><td>• 【符号扩展】<br>○ 正数符号位不变，扩展位填充0<br>○ 负数原码表示和正数相同，符号位为1<br>○ 负数补码符号位不变，附加位用1(对于整数)或0(对于小数)进行填充<br>○ 如16位补码Ox8FA0扩展位32位为OxFFFF 8FA0<br>○ 如−64位的补码是1100 0000，则其十六进制为OxFFFF FFC0<br>• 该方法商符和商值分开进行<br>• 商符由两个操作数的符号位异或形成</td><td><br>【例 2.9】设 x = 0.1011，y = 0.1101，采用原码加减交替除法求 x/y。<br>解：|x| = 0.1011，|y| = 0.1101，[|y|]补 = 0.1101，[−|y|]补 = 1.0011。<br>原码不恢复余数法的求解过程如下。<br>被除数 商 说明<br>0.1011 0.0000 起始情况<br>+[−|y|]补 1.0011 −|y|即+[−|y|]补<br>1.1110 0.0000 部分余数为负，商上0<br>左移 1.1100 -------- 0.0000 余数和商左移一位<br>+|y| 0.1101 +|y|<br>0.1001 0.0001 部分余数为正，商上1<br>左移 1.0010 -------- 0.0010 余数和商左移一位<br>+[−|y|]补 1.0011 −|y|即+[−|y|]补<br>0.0101 0.0011 部分余数为正，商上1<br>左移 0.1010 -------- 0.0110 余数和商左移一位<br>+[−|y|]补 1.0011 −|y|即+[−|y|]补<br>1.1101 0.0110 部分余数为负，商上0<br>左移 1.1010 -------- 0.1100 余数和商左移一位<br>+|y| 0.1101 +|y|<br>0.0111 0.1101 部分余数为正，商上1<br>余数 商<br>因此 $Q_s = x_s \oplus y_s = 0 \oplus 0 = 0$，得 x/y = +0.1101，余 $0.0111\times2^{-4}$。</td><td>

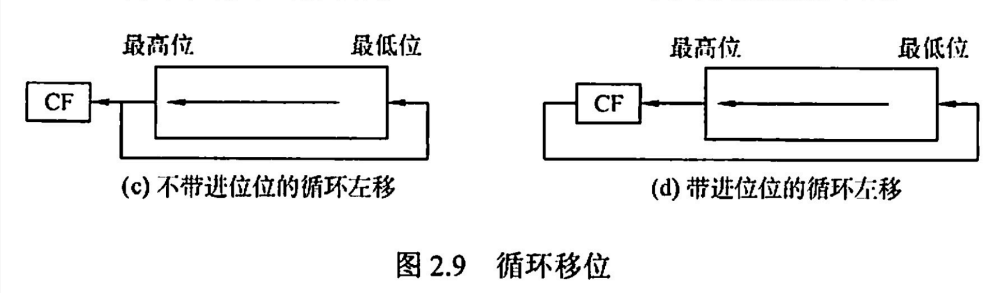
除数寄存器 Y<br>32<br>32<br>32位 ALU<br>加/减<br>32<br>64位<br>余数寄存器 R<br>余数/商寄存器 Q<br>左移<br>写使能<br>控制逻辑 计数器$C_n$<br>32<br>时钟

<br>图 2.13 32 位除法运算的逻辑结构图</td></tr>
<tr><td>补码除法运算【加减交替法】</td><td>• 该方法符号位和数值位一起参加运算，商符自然形成<br>• step1：根据被除数和除数的符号决定做加法还是减法<br>• step2：上商的原则根据余数和除数的符号位共同决定<br>○ 同号上商1<br>○ 异号上商0<br>• step3：最后一步商置1</td><td><br>【例 2.10】设 x = 0.1000，y = −0.1011，采用补码加减交替法求 x/y。<br>解：采用两位符号表示，[x]原 = 00.1000，则[x]补 = 00.1000。[y]原 = 11.1011，则[y]补 = 11.0101，[−y]补 = 00.1011。补码加减交替法的求解过程如下。<br>被除数 商 说明<br>00.1000 0.0000 起始情况<br>+[y]补 11.0101 [x]补、[y]补异号，则+[y]补<br>11.1101 0.0001 部分余数与[y]补同号，则商上1<br>左移 11.1010 -------- 0.0010 左移一位<br>+[−y]补 00.1011 +[−y]补<br>00.0101 0.0010 部分余数与[y]补异号，则商上0<br>左移 00.1010 -------- 0.0100 左移一位<br>+[y]补 11.0101 +[y]补<br>11.1111 0.0101 部分余数与[y]补同号，则商上1<br>左移 11.1110 -------- 0.1010 左移一位<br>+[−y]补 00.1011 +[−y]补<br>00.1001 0.1010 部分余数与[y]补异号，则商上0<br>左移 01.0010 -------- 1.0100 左移一位<br>+[y]补 11.0101 +[y]补<br>00.0111 1.0101 末位恒置1<br>余数 商<br>所以有[x/y]补 = 1.0101，余 $0.0111\times2^{-4}$。</td><td>

除数寄存器 Y<br>32<br>32<br>32位 ALU<br>加/减<br>32<br>64位<br>余数寄存器 R<br>余数/商寄存器 Q<br>左移<br>写使能<br>控制逻辑 计数器$C_n$<br>32<br>时钟

<br>图 2.13 32 位除法运算的逻辑结构图</td></tr>
</table>

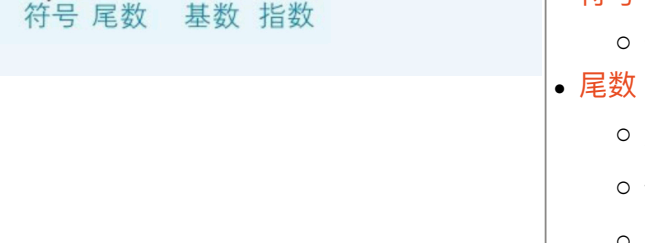
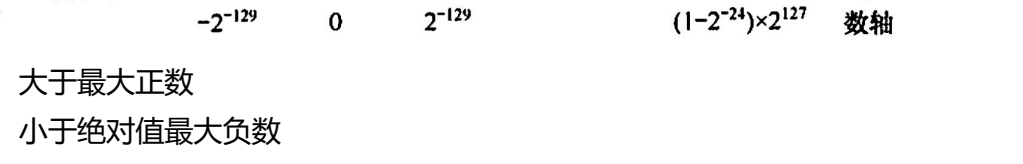
### 移位计算

<table>
<tr><td rowspan="3">移位运算</td><td>算术移位</td><td>• 算术移位的对象是有符号数，移位过程中符号位保持不变<br>• 算术左移情况下，补码左移的前提条件是其原最高有效位与原符号位要相同【如符号位1，最高有效位1，数据不丢失】<br>• 算术左移实现乘法功能，算术右移实现除法功能<br>• 例：十进制 $-4\ [11111100]_补$ 右移两位 −−高位填充符号位 −−→ $[11111111]_补$ 即十进制 −1，是 −4的$\frac{1}{4}$<br>
<table>
<tr><th></th><th>码制--->添补规则</th></tr>
<tr><td>正数+</td><td>原，反，补---->0</td></tr>
<tr><td>负数-</td><td>原码--->0<br>补码左移--->0<br>补码右移--->1<br>反码--->1</td></tr>
</table>
</td></tr>
<tr><td>逻辑移位</td><td>• 逻辑移位视数为无符号数<br>• 左移，低位补0<br>• 右移，高位补0<br>• 例：十进制 $-4\ [11111100]_补$ 右移两位 −−高位补0 −−→ $[00111111]_补$ 即十进制63，不是 −4的$\frac{1}{4}$</td></tr>
<tr><td>循环移位</td><td>【分类】带进位标志CF的循环移位（大循环）+ 不带进位标志位CF的循环移位（小循环）<br>【主要特点】<br>• 移出的数位又被移入数据中，而是否带进位则要看是否将进位标志符加入循环位移<br>• 循环移位适合将数据的低字节数据和高字节数据互换<br>
<table>
<tr><td>• d中，数据位连同CF一起左移<br>• 数据的最高位移入CF<br>• CF则移入数据的最低位</td><td>

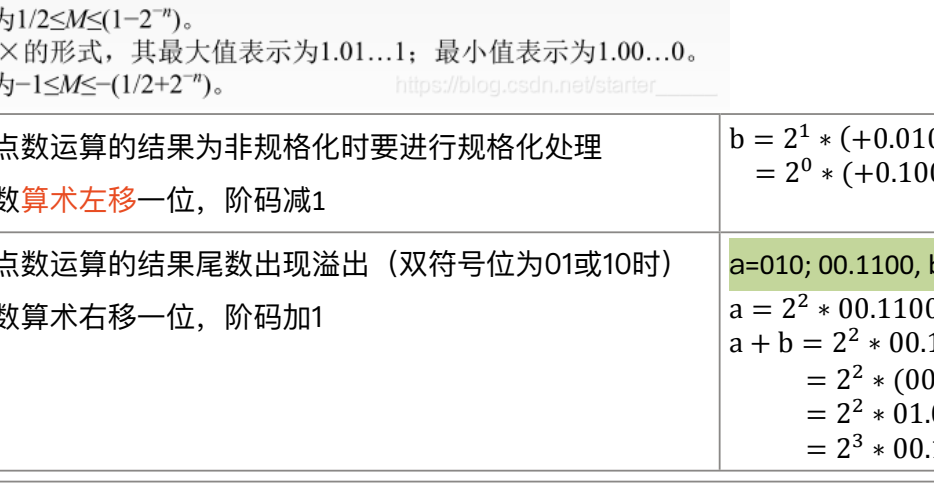
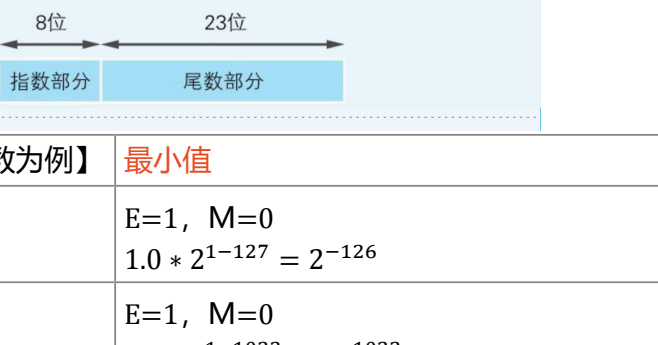
最高位 最低位<br>CF<br>(a) 不带进位位的循环右移<br>(b) 带进位位的循环右移<br>(c) 不带进位位的循环左移<br>(d) 带进位位的循环左移

<br>图 2.9 循环移位</td></tr>
</table>
</td></tr>
</table>

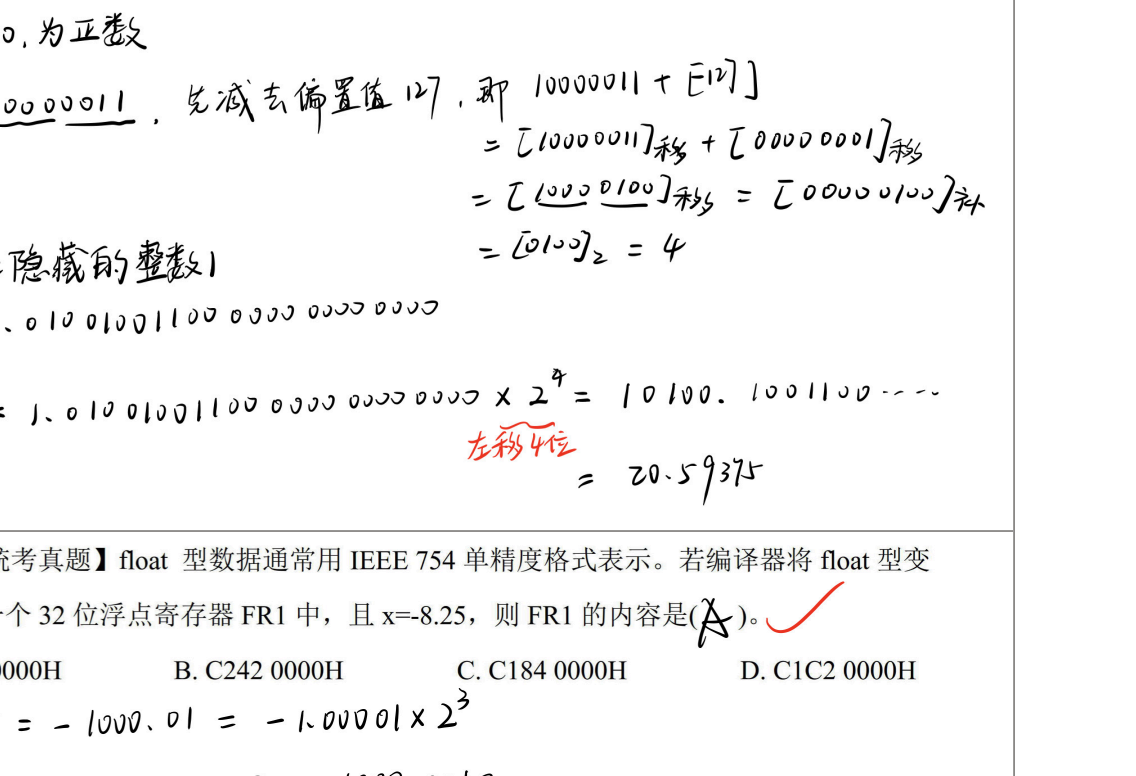
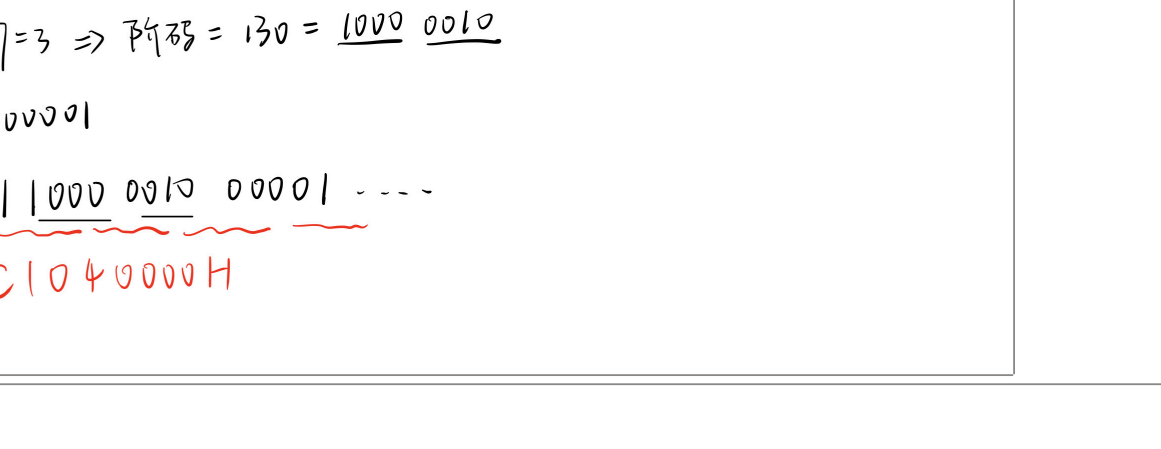
## 浮点数【对照十进制的科学计数法！！！】

<table>
<tr><td>为什么要有浮点数?</td><td>浮点数是什么?</td></tr>
<tr><td>• 由于二进制表示小数会出现问题，如不能用二进制表示0.1，所以设计了浮点数来表示小数。<br>• 浮点数表示法能以适当的形式将比例因子表示再数据中，让小数点的位置根据需要而浮动。<br>• 在位数有限的情况下，既扩大了数的表示范围，又保持了数的有效精度</td><td>• 浮点数是指用符号，尾数，基数和指数来表示的小数<br>• 浮点数有两种【双精度浮点数double, 64位】【单精度浮点数float, 32位】<br>
<table>
<tr><td> </td><td>• 基数<br>○ 二进制基数为2<br>○ 基数越大，表示范围越大，但是精度越低<br>• 符号<br>○ 指使用一个数据位来表示数值的符号，1为负，0为正<br>• 尾数<br>○ 尾数的位数反映浮点数的精度<br>○ 使用正则表达式、将表现形式多样的浮点数统一为一种表达式<br>○ 规则：基数为2时，将小数点前面的值固定为1；基数为4时，尾数的最高两位不全为0<br>• 指数(阶码)部分<br>○ 阶码的位数反映浮点数的表示范围</td></tr>
</table>
</td></tr>
<tr><td>浮点数的表示范围</td><td>浮点数举例</td></tr>
<tr><td>

可表示的负数<br>零<br>可表示的正数<br>负上溢<br>负下溢<br>正下溢<br>正上溢<br>$-(1-2^{-24})\times2^{127}$<br>$-2^{-129}$<br>0<br>$2^{-129}$<br>$(1-2^{-24})\times2^{127}$<br>数轴

<br>• 【正上溢】：大于最大正数<br>• 【负上溢】：小于绝对值最大负数<br>• 【正下溢】：0到最小正数之间<br>• 【负下溢】：0到绝对值最小负数之间<br>• 通俗来说，上溢就是大于最大数，下溢就是小于最小数</td><td>阶码，位数均用补码表示，求a,b的真值<br>
<table>
<tr><td>a=001;1.1001;</td><td>001对应值为1，即阶码为1，尾数1.1001对应值$-\frac{7}{16}$<br>$a = 2^1 * \left(-\frac{7}{16}\right) = -\frac{7}{8}$</td></tr>
<tr><td>b=001;0.01001</td><td>001对应值为1，即阶码为1，尾数0.01001对应值$+\frac{9}{32}$<br>$b = 2^1 * \left(\frac{9}{32}\right) = \frac{9}{16}$</td></tr>
</table>
</td></tr>
<tr><td>规格化浮点数</td><td>浮点数加减运算过程【选择题直接用数学知识计算】</td></tr>
<tr><td>• 规格化：规定尾数的最高数位必须是一个有效值(1)<br>• 采用规格化浮点数的目的是为了增加数据的表示精度<br><br>1. 原码规格化后：<br>正数为0.1××…×的形式，其最大值表示为0.11…1；最小值表示为0.10…0。<br>尾数的表示范围为$1/2 \leq M \leq (1-2^{-n})$。<br>负数为1.1××…×的形式，其最大值表示为1.10…0；最小值表示为1.11…1。<br>尾数的表示范围为$-(1-2^{-n}) \leq M \leq -1/2$。<br>2. 补码规格化后：<br>正数为0.1××…×的形式，其最大值表示为0.11…1；最小值表示为0.10…0。<br>尾数的表示范围为$1/2 \leq M \leq (1-2^{-n})$。<br>负数为1.0××…×的形式，其最大值表示为1.01…1；最小值表示为1.00…0。<br>尾数的表示范围为$-1 \leq M \leq -(1/2+2^{-n})$。<br>
<table>
<tr><td>左规</td><td>• 当浮点数运算的结果为非规格化时要进行规格化处理<br>• 将尾数算术左移一位，阶码减1</td><td>$b = 2^1 * (+0.01001)$<br>$= 2^0 * (+0.10010)$</td></tr>
<tr><td>右规</td><td>• 当浮点数运算的结果尾数出现溢出（双符号位为01或10时）<br>• 将尾数算术右移一位，阶码加1</td><td>a=010; 00.1100, b=010; 00.1000, 求a+b<br>$a = 2^2 * 00.1100; b = 2^2 * 00.1000$<br>$a + b = 2^2 * 00.1100 + 2^2 * 00.1000$<br>$= 2^2 * (00.1100 + 00.1000)$<br>$= 2^2 * 01.0100$<br>$= 2^3 * 00.1010$</td></tr>
</table>
</td><td>
<table>
<tr><td>对阶</td><td>• 目的：使两个操作数的小数点位置对齐<br>• 对阶操作是把较小的阶码调整到较大的阶码<br>• 阶码增大，尾数右移<br>• 无阶码减小的情况</td></tr>
<tr><td>尾数求和</td><td>• 对尾数进行加减法</td></tr>
<tr><td>规格化</td><td>• 规格化浮点数</td></tr>
<tr><td>舍入</td><td>• 舍入是浮点数概念，定点数无舍入<br>• 浮点数舍入的情况：对阶或者右规格化<br>• 舍入不一定产生误差（后几位为0时不产生误差）</td></tr>
<tr><td>溢出判断</td><td>• 如果双符号位为01或10时，则溢出</td></tr>
</table>
</td></tr>
<tr><td>C语言中的浮点数类型</td><td>IEEE 754浮点数的范围【常考选择题】</td></tr>
<tr><td>• 以下转换的范围和精度都从小到大，转换过程没有损失<br>○ char-->int-->long-->double<br>○ float-->double<br>• int/float-->double【能保留精确值】<br>• int-->float【不会发生溢出，int值过大的话可能会发生舍入】<br>• double-->float【可能会发送溢出和舍入】<br>• float/double-->int【可能会发生溢出和舍入】</td><td>• IEEE 754中尾数用原码表示，阶码用移码表示<br>

●双精度浮点数（共64位）<br>1位<br>11位<br>52位<br>符号部分<br>指数部分<br>尾数部分<br>●单精度浮点数（共32位）<br>1位<br>8位<br>23位<br>符号部分<br>指数部分<br>尾数部分

<br>
<table>
<tr><th>【以正数为例】</th><th>最小值</th><th>最大值</th></tr>
<tr><td>单精度</td><td>E=1，M=0<br>$1.0 * 2^{1-127} = 2^{-126}$</td><td>E=254，M=.111..., 1.111...1$* 2^{254-127} = 2^{127} * (2-2^{-23})$</td></tr>
<tr><td>双精度</td><td>E=1，M=0<br>$1.0 * 2^{1-1023} = 2^{-1022}$</td><td>E=2046，M=.111..., 1.111...1$* 2^{2046-1023} = 2^{1023} * (2-2^{-52})$</td></tr>
<tr><td>死记硬背</td><td>单精度阶码取值1~254，偏移值为127<br>最小值为$1 * 2^{1-127}$<br>最大值为$(2-2^{23}) * 2^{254-127}$</td><td>双精度阶码取值1~2046，偏移值为1023</td></tr>
<tr><td>理解1</td><td>规定除去E为全0和全1表示0和无穷大的情况，所以阶码范围为1~254</td><td></td></tr>
</table>
</td></tr>
<tr><td>计算浮点数对应的十进制【例题】【常考选择题】</td><td></td></tr>
<tr><td>• 单精度中，最高位为符号位，接着的8位为阶码，偏置值为127，后23位为尾数值【注意，尾数值隐藏了整数部分1】<br>• 双精度中，最高位为符号位，接着的11位为阶码，偏置值为1023，后52位为尾数值<br>
<table>
<tr><td>将浮点数转为十进制</td><td><br>10.按照 IEEE 754 标准规定的 32 位单精度浮点数 41A4C000H 对应的十进制数是（D）。<br>A. 4.59375　B. -20.59375　C. -4.59375　D. 20.59375<br>41A4C000H = 0100 0001 1010 0100 1100 0000 0000 0000<br>符号位 0　8位，阶码[移码]　尾数[原码]<br>①符号位为0，为正数<br>②阶码：10000011，先减去偏置值127，即 10000011 + [−127]<br>= [10000011]移 + [00000001]移<br>= [10000100]移 = [00000100]补<br>= $[0100]_2$ = 4<br>③尾数加上隐藏的整数1<br>1.01001001100 0000 0000 0000<br>∴最后数 = 1.01001001100 0000 0000 0000 × $2^4$ = 10100.1001100····<br>左移4位<br>= 20.59375</td></tr>
<tr><td>将十进制转为浮点数</td><td><br>24.【2011 统考真题】float 型数据通常用 IEEE 754 单精度格式表示。若编译器将 float 型变量 x 分配在一个 32 位浮点寄存器 FR1 中，且 x=-8.25，则 FR1 的内容是（A）。<br>A. C104 0000H　B. C242 0000H　C. C184 0000H　D. C1C2 0000H<br>x = −8.25 = −1000.01 = −1.00001 × $2^3$<br>∴阶码 −127 = 3 ⇒ 阶码 = 130 = 1000 0010<br>尾数 = 1.00001<br>∴FR1 = 1 1000 0010 00001 ····<br>= C1040000H</td></tr>
</table>
</td><td></td></tr>
</table>

## 定点数和浮点数的区别

| | 定点数 | 浮点数 |
|---|---|---|
| 数值的表示范围 | | 更大 |
| 精度 | 更高 | |
| 数的运算 | | 更复杂 |
| 溢出问题 | 运算结果超出数的表示范围则溢出 | 运算结果超出数的表示范围不一定溢出 |